

INDIVIDUAL MODULE-WISE LAB WORKBOOK

EXPERIMENT 4

Module: Robot Framework

Student Name: Puspendu Sekhar Das

Enrollment Number: 12023002022220

Department: CST

Institution: Institute of Engineering and Management, Newtown

Experiment: Robot Framework

Module Name and Experiment Title

Module: Robot Framework

Title: Web Test Automation Using Robot Framework and SeleniumLibrary

Problem Statement

Automation scripts can become difficult to maintain when test actions, setup, verification and execution commands are mixed together. Robot Framework provides a keyword-driven approach in which readable test cases can be built from standard and user-defined keywords. This experiment demonstrates browser automation, variables, data-driven tests, custom keywords, setup/teardown, tags, command-line execution and HTML reporting.

Objective

- Understand the purpose and structure of Robot Framework.
- Configure Robot Framework and SeleniumLibrary for browser automation.
- Create readable test cases using keywords and variables.
- Use setup and teardown, command-line options and tags.
- Create user-defined keywords and organize reusable keywords in resource files.
- Perform browser automation using SeleniumLibrary.
- Generate Robot Framework output, logs and HTML reports.

Tools, Software and Concepts Used

Category	Details
Framework	Robot Framework
Browser Library	SeleniumLibrary
IDE / Tools	RIDE / PyCharm / Visual Studio Code / Windows Command Window
Test Style	Keyword-driven and readable test cases
Execution	robot command, suite/test selection, tags and run options
Design	User-defined keywords, resource files, setup and teardown, page-object concepts
Reporting	output.xml, log.html and report.html

Implementation Steps

- Install Robot Framework and SeleniumLibrary and configure the working environment.
- Create a .robot test file with settings, test cases and keywords.
- Open a browser, navigate to a URL and verify page content.
- Use variables and data-driven test cases for repeated inputs.
- Create user-defined keywords and move reusable keywords into resource files.
- Add suite/test setup and teardown and use tags for selective execution.
- Run tests from the command line and inspect generated HTML reports and logs.

Source Code / Representative Implementation

```
*** Settings ***
Library      SeleniumLibrary
Suite Setup   Open Browser      https://example.com    chrome
Suite Teardown  Close All Browsers

*** Test Cases ***
Verify Example Page
    Title Should Be      Example Domain
    Page Should Contain   Example Domain
```

Robot Code

```
1 *** Settings ***
2 Library      SeleniumLibrary
3 Suite Setup   Open Browser      https://example.com    chrome
4 Suite Teardown  Close All Browsers
5
6 *** Test Cases ***
7 Verify Example Page
8     Title Should Be      Example Domain
9     Page Should Contain   Example Domain
```

Figure 1: Robot Framework test case using SeleniumLibrary

Key Implementation Concepts

- Robot Framework uses keywords to represent browser and verification actions.
- SeleniumLibrary provides browser automation keywords such as Open Browser and Page Should Contain.
- Variables can store usernames, passwords and other reusable test data.
- User-defined keywords improve readability and allow common actions to be reused.
- Suite Setup and Suite Teardown provide common initialization and cleanup.
- Tags can be used to include or exclude selected tests during command-line execution.
- Resource files allow reusable keywords and variables to be shared between suites.
- Robot Framework generates execution artifacts including output.xml, log.html and report.html.

Windows PowerShell

```
PS D:\Documents\Wipro Lab Reports> robot tests
=====
Verify Example Page                                     | PASS |
-----
1 test, 1 passed, 0 failed
Output:  D:\Documents\Wipro Lab Reports\output.xml
Log:     D:\Documents\Wipro Lab Reports\log.html
Report:  D:\Documents\Wipro Lab Reports\report.html
```

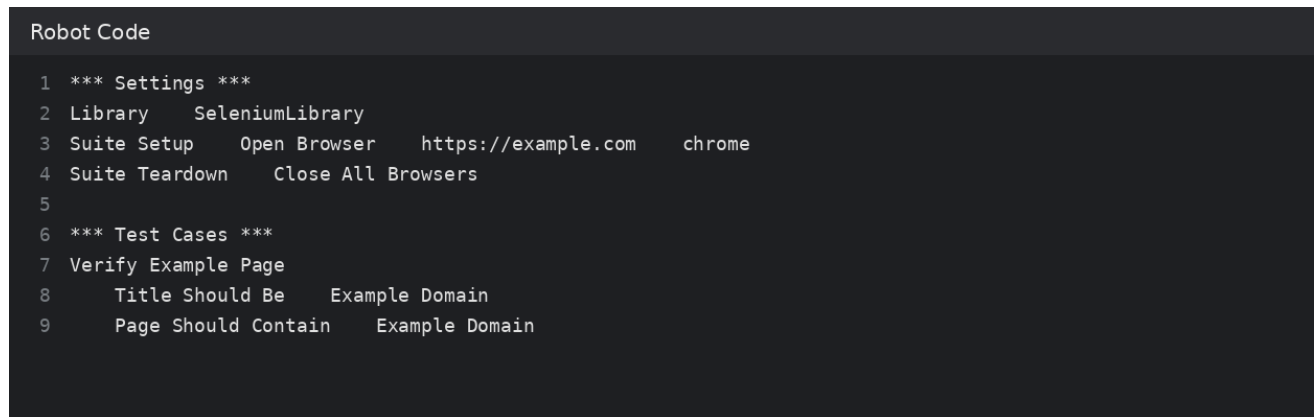
Figure 2: Robot Framework execution from the laboratory terminal

Script Running and Readability

Robot Framework tests can be executed from PyCharm, a command window, batch files or scheduled environments. The framework also supports breaking a procedural script into readable user-defined keywords and moving reusable keywords to resource files.

```
*** Keywords ***
Open Test Page
    Open Browser    https://example.com    chrome

Verify Example Page
    Title Should Be    Example Domain
    Page Should Contain    Example Domain
```

A screenshot of a code editor window titled "Robot Code". The code is written in a dark-themed editor with line numbers on the left. The code defines two keywords: "Open Test Page" and "Verify Example Page".

```
1  *** Settings ***
2  Library    SeleniumLibrary
3  Suite Setup    Open Browser    https://example.com    chrome
4  Suite Teardown    Close All Browsers
5
6  *** Test Cases ***
7  Verify Example Page
8      Title Should Be    Example Domain
9      Page Should Contain    Example Domain
```

Figure 3: User-defined keywords for improving Robot Framework readability

Output with Clearly Labelled Screenshots

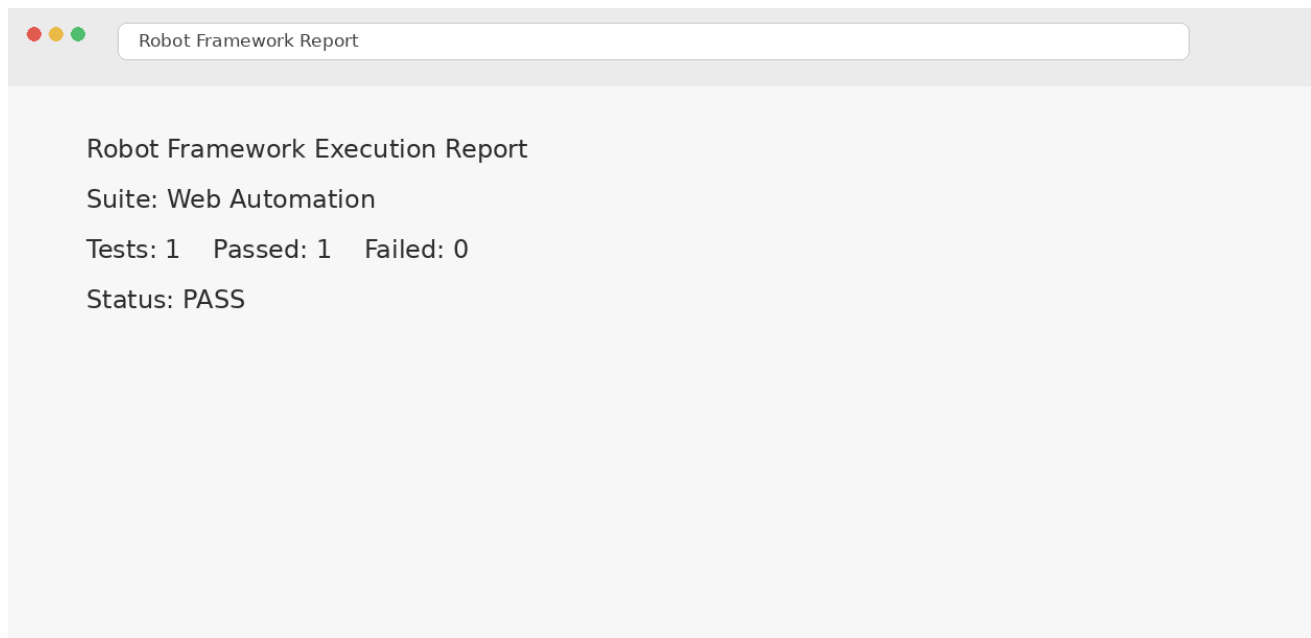


Figure 4: Robot Framework execution report

Result

The Robot Framework automation workflow was organized using SeleniumLibrary and keyword-driven test cases. Browser navigation, verification, setup/teardown and report generation were demonstrated through a compact Robot Framework suite.

Observations

- Keyword-driven syntax makes automation steps readable and easy to organize.
- SeleniumLibrary provides reusable browser interaction keywords.
- Variables and user-defined keywords reduce duplication in test cases.
- Tags and command-line options provide control over which tests are executed.
- Robot Framework automatically produces structured execution logs and HTML reports.

Conclusion

The Robot Framework module was completed through practical browser automation and framework organization. The experiment provided hands-on understanding of keyword-driven testing, SeleniumLibrary, variables, reusable keywords, setup/teardown, test execution and reporting.